The Relay

The code is in `relay/`. `results/results.csv` has one row per request: 797 rows, 219 against the real service and 578 against the stand-in. The columns after `answer_grade` are extras (model, cache hit, degrade level, shed reason, wait/work/retry ms, calls). Per-request traces, including every logged draft, are in `results/traces.jsonl`, and the raw probes of the real service are in `results/probe_raw.jsonl`. `python -m relay.finalize && python -m relay.analyze results/results.csv` rebuilds every table below.

1 What I built and why

Five steps: gpt-oss-20b classifies the question (in scope, which paper, search words); a BM25 tool retrieves 250-word pieces (1.5 s timeout, in-process fallback); a branch refuses early if nothing relevant comes back, or picks the model and passage count from the remaining budget; gpt-oss-120b drafts a streamed answer citing passages as [n]; a deterministic checker shows a sentence only if it cites a passage it was given and every number in it appears there, redrafting once and otherwise refusing.

The decision that shaped it most: tokens per minute, not requests, are the scarce resource. Each model allows 8,000 TPM and a real answer costs 2,174 tokens, about 3.5 answers a minute. So the checker is code, not a model call; out-of-scope questions are refused before drafting; repeats come from a cache; and a governor tells anyone it can't serve within 20 s straight away. Fast means the first verified sentence within 2 s at p50 and 10 s at p95 when not overloaded; holding sentences until checked costs time to first token, which I accept.

2 The real service and my stand-in

165 real calls (latency samples, 40 at once, back-to-back 697-token prompts, 12 concurrent) gave Table 1. Groq's limits refill continuously: 2.7 s after a per-minute 429 a call succeeded; the token reset grew 5.2 s per 697 tokens (8,000/60 s); the daily reset grew 86.4 s per call; the daily-token 429 asked for 696 s. So I rebuilt the governor as refilling buckets.

The stand-in (`relay/standin.py`) streams Groq-shaped replies with lognormal first-token latency (median 469 ms, σ 0.61 for 120b; 500 ms, σ 0.29 for 20b), 1,222 and 2,190 tokens/s, 4% slow successes at 12×, 0.5% early and 0.3% mid-stream failures, Groq-style 429s and refilling buckets, and slowdown past 8 calls in flight. It is least like Groq in four ways: its median is too optimistic (730 ms vs 1,944 ms real, because real reasoning takes longer); the load slowdown is invented (12 real concurrent calls showed none); its answers echo passage 1, so it never measures quality; and it never cites as 【n】.

3 Results

Percentiles are nearest-rank over rows with status ok, with p95 and p99 as the slow end; "Requests sent" gives n. Only `steady-real` and the quality runs used the real service, and the rest used the stand-in. Cache hits count as served answers. The cache served 15% of requests in the steady run, 29% in the burst and 38% in the overload.

The number I could not have guessed: fixing how the checker reads citation brackets took gpt-oss-120b from 40% to 60% correct and cut its first-token p95 from 12,862 ms to 3,082 ms. Each wrong refusal had triggered a redraft, and each redraft waited for token budget.

On my 25 questions (20 answerable, 5 not), the final configuration got 15 right, none wrong, refused all 5 unanswerable ones and refused 5 answerable ones. Across all 175 real graded answers, 3 were wrong and all three passed the checker. In each, the number really was in the cited passage, attached to the wrong thing: "Fisher … 170 hours" (the 170 h belongs to a different dataset), and two h6 answers describing the Chinese-model training set. The number check catches invented numbers; it can't tell when a real number is pinned to the wrong noun.

The five wrong refusals each have a different cause. For m3 and m7, the understand step doesn't know Helium is Moshi's language model. For h4, the model worked out "6.4 points", which the paper never prints. For h6, retrieval missed the passage. For x1, the answer sits in a table that PDF extraction scrambles. Answers used 1,854 to 2,668 tokens depending on the setting.

4 Where the time goes

Request `steady-real-0003` (real; "…Moshi's theoretical latency and its latency in practice?") took 13,583 ms to its first token, and all of the delay came before it. The trace times each call:

Part	Time	What happened
Working	798 ms	Understand call
Retrying	697 ms	First draft, rejected as an "uncited sentence" (the 【n】 bug)
Waiting	11,564 ms	The governor held the redraft until the 120b TPM bucket refilled (`wait_ms` in the row)
Working	521 ms	Second draft, rejected again; the system then refused

That adds up to about 13.6 s, and it is how I found that a checker bug shows up as latency.

5 When a limit is hit

At the per-minute limit, requests already in progress keep their place, so a redraft goes ahead of new arrivals. A new arrival whose predicted wait is longer than its 20 s budget is told so within milliseconds. If the wait for 120b passes 10 s, the request skips the understand step and drafts on 20b with 2 passages, using that model's separate 8K TPM bucket. When limits change without notice, the governor picks up the new number from Groq's `x-ratelimit-limit-tokens` header and from the "Limit N" in the 429 text. With limits halved, 31.0% of people gave up when the governor ignored the change and 6.9% when it adopted it.

At the per-day limit, the 120b bucket empties first and new answers move to 20b. Once both are empty, the person gets the reply in §7 in about 1 ms, and cached answers keep flowing. I chose to turn people away instead of queueing them for minutes, because the brief says people give up sooner than expected.

Retrieval: quality versus cost

All runs were real, 25 questions each, on gpt-oss-20b with the first version of the checker (Table 3).

Setting	Correct
120 words × 8, re-rank	52%
250 × 4, re-rank	60%
250 × 4, no re-rank	64%
500 × 2, no re-rank	60% (fewest tokens: 1,854)

With and without re-rank differ by one question, which is within noise. Small pieces scored lowest, and I didn't check why. I'd ship 250 × 4 without re-rank because it scored best and is simpler; the 120b runs used re-rank, and I would turn it off. Near the limit, the system drops to 250 × 2, about 720 context tokens instead of 1,500. Token-free recall stays at 90% there, but I didn't measure answer quality at that setting.

How this library will mislead a naive retriever

First, the papers contradict themselves and the abstract ranks first. AIAYN's abstract gives 41.8 BLEU for English-French while §6.1 says 41.0, and H-Net's abstract says DNA data efficiency improves "nearly 4×" while its introduction says 3.6×. Ask "What BLEU did the Transformer get on English-French?" and the abstract comes back first.

Second, the papers share vocabulary. "Heads", "Transformer", "tokens" and "chunk" appear in all three. "How many heads does Moshi's Temporal Transformer use?" pulls AIAYN's h = 8 and Mimi's 8 heads, while the answer, 32, sits in a table that pdftotext scrambles. Both models refused it, which was correct given what they were shown.

Third, questions use different words from the papers. "Which dataset were the H-Net LMs trained on?" missed "100B token subset sampled from FineWeb-Edu" in every setting. In the other direction, "What BLEU does Moshi get on WMT14?" got the highest retrieval score of all 25 questions (51.7), so the refusal has to come from the model and the checker.

6 Retries

Each call gets at most 2 retries. The system honours `retry-after`, and otherwise backs off 0.5 s and then 1 s with jitter. It retries only if the wait still

fits the deadline, and never retries a call that has already streamed tokens.
Retrying made things worse on the stand-in with limits halved and the governor unaware. Naive retries (3 tries, 0.2 s apart, ignoring `retry-after`) pushed first-token p95 from 3,819 ms to 20,181 ms. In `halved-static-naive-0018`, the draft got a 429, the immediate retry joined the back of the queue and waited 18.2 s, and the person gave up at 19.8 s. Retrying helped on the real service in `steady-real-0007`, which hit a dropped connection, retried and answered in 2,000 ms. In normal overload, naive retries barely fire (0.02 per request), because the governor prevents 429s in the first place. Retries matter when the governor's picture of the limits is wrong.

7 What the person sees

At 3pm, with the daily budget gone, this reply arrives in about 1 ms:

> *"Today's model budget is used up, so I can't write a new checked answer right now. Capacity comes back gradually; try again in about N min. Nobody has asked this exact question today, so there is no saved answer to show you. Meanwhile, these are the passages most likely to answer it. They are raw excerpts, not an answer, and I have not checked them against your question: [paper, page: excerpt] ×3."*

N comes from the measured refill rate of one request per 86 s. The reply never says "tomorrow", because Groq's daily budget refills continuously. Anyone asking a question already answered today gets that checked answer instead.

8 What I'm unsure about, what I left out, what I learned

My least certain assumption is how patient people are: lognormal with a 12 s median. Every shed threshold rests on it. If people wait longer, I'm turning away people I could have served. If they give up sooner, the 20 s budget is already too long; in the stand-in overload run, 11 people gave up while still in the queue.
I deliberately left out two things. An LLM judge would roughly double tokens per answer under an 8K TPM ceiling. Embeddings and re-ranking models weren't worth extra calls for three papers and mostly lexical questions.
I changed one rubric after reading answers, and I'm disclosing it. Question m6 was ambiguous, because the paper fine-tunes on Fisher (2,000 h) and also on a 170 h "supervised multi-stream dataset". Answers that give Fisher 170 h are still graded wrong. Every regrade is listed in `regrade_log.csv`.
New to me: I had assumed "per-minute" and "per-day" limits reset on the clock, at the top of the minute or at midnight. Groq's don't. They work more like a bucket that refills steadily: 8,000 tokens a minute is really about 133 tokens every second, and 1,000 requests a day means one request comes back every 86 seconds. I saw this in the response headers. The daily reset timer went up by 86.4 s with each request, and after I hit the minute limit I was let back in after 2.7 s, not 60. That changed my design. There is no reset moment to wait for, and a message telling someone to "come back tomorrow" would simply be wrong.
I changed three things: the wording is first-person and conversational, the "bucket" idea is introduced before the numbers, and the closing line now says why it matters. If I need it shorter for the 2-page limit, I'll cut the 133 tokens/second example and keep the 86.4 s and 2.7 s observations, since those are my evidence.

Your feedback on Vaani

The "zero dead air" and turn-taking ideas stood out: the team cares how a call feels, and the ~450 ms p90 latency claim and customer logos back it up. "The only voice AI harness you'll ever need" confused me, as did start buttons pointing to two places (app.vaanivoice.ai and platform.vaani.dev). I'd add one plain line under the headline saying what Vaani does and for whom, next to a single "Start free" button.

How I worked with the assistant

What I did (decided and directed)

- I opened with a full brief and hard rules:
  - never invent a number; every figure is computed from `results.csv` or the raw logs
  - ask me before any run that spends Groq quota
  - log raw headers and bodies, including every 429
  - keep a `DECISIONS.md`
  - work in gated stages
- I kept my API key in my own shell and never pasted it. The assistant could only check that it existed (True/False).
- I defined "fast":
  - first token means the first token of answer text; status messages don't count
  - first-token p95 ≤ 5 s, total p95 ≤ 12 s, and a person gives up after 20 s with no answer token
- I decided to write about 20 questions myself, with gold answers, and told the assistant not to write or suggest any.
- I chose hand-grading over an LLM judge.
- I set the quota rules: never deliberately exhaust the daily limit, and ask before any run over 25% of daily requests.
- I set the citation format and asked for tests under `tests/`, diagnostic output in the fetcher, a secret scanner, and a log entry for the assistant's shell-escaping bug.
- I chose gpt-oss-20b for the probe, with reasoning tokens timed separately and low reasoning effort.
- I approved a Day 1 probe of at most 100 requests, with safeguards:
  - a dry-run mode
  - a hard request cap
  - stop after 3 unexpected errors
  - no automatic retries

What the assistant produced

- A restated plan with flags:
  - tokens per minute would be the binding limit
  - stand-in answers can't be graded meaningfully
  - the budget-exhausted scenario should run on the stand-in
  - p99 from a small probe is low confidence
- A repo skeleton: `check_env.py`, `fetch_papers.py`, a PyMuPDF extractor, and `DECISIONS.md` (D1–D13, plus incident notes I1 and I2).
- A secret scanner that prints only file, line and rule, with a `--history` mode.
- Unit tests: 7, later 12, all passing.
- A summary of Groq's documented limits and headers.
- A two-day probe plan with request and token costs, each day under my 25% rule.

Three prompts that mattered

1. The opening brief with the hard rules. It set how the assistant had to behave for the whole chat.
2. "Plan approved…". This fixed the definition of "fast", my-questions-only, hand-grading and the quota rules.
3. The Day 1 probe approval. It set the safety limits on spending real quota.

What it got wrong that I caught

In that chat I didn't catch an error myself. The assistant reported its own mistakes: a shell-escaping bug in its first scripts, and a directory walker too clever to explain. I made it log both in `DECISIONS.md` so they'd show in the history. If you need a real catch, the 404 run and the 【3】 citation bug are in this chat.

Where I overruled it

- Citations (D11): the assistant planned to cite PDF page numbers. I changed it to paper + nearest section heading + PDF page, plus a manifest field recording the offset between PDF and printed page numbers.
- Model choice: the assistant wanted to pick the model after a first probe. I chose gpt-oss-20b up front and set the conditions.
- 12 s shed threshold: when I asked for it, the assistant said the code didn't exist there. I dropped it and required it later as a named, tested constant chosen from probe data. That one was the assistant pushing back, not me overruling it.

Results tables

Table 1: Limits found in the real service

Limit	Value found	How I found it	Confidence
Requests per minute	30 per model	40 calls at once: 31 ok, 9 × 429 “on requests per minute (RPM): Limit 30, Used 30”	high
Tokens per minute	8,000 per model	`x-ratelimit-limit-tokens` on every response; refills about 133/s (reset grew 5.2 s per 697 tokens)	high
Requests per day	1,000 per model	`x-ratelimit-limit-requests`; reset header grows 86.4 s per call (rolling)	high
Tokens per day	200,000 per model	Groq rate-limit docs, plus a real 429 with retry-after 696 s, consistent with a 200K/day refill (that 429’s body was not logged)	medium
Requests in flight at once	no limit seen at 12	12 concurrent calls to 120b, all 200	low
What it returns when a limit is hit	HTTP 429, JSON `error.type = requests/tokens`, `code = rate_limit_exceeded`, message naming the limit and “try again in Xs”; headers `retry-after` and `x-ratelimit-*`	captured in `probe_raw.jsonl`	high
Time until it recovers	RPM: 2.7 s; daily tokens: 696 s for one ~1.6K-token call	polling every 2 s after the 429; retry-after on the TPD 429	medium

Table 2a: Latency per load scenario

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
steady-real	real	11	299	2.3	1944 / 13583 / 13583	1944 / 13583 / 13583
steady	standin	27	234	7.6	730 / 7591 / 9517	744 / 9534 / 20836
burst	standin	66	214	19.7	457 / 10300 / 12628	487 / 10313 / 12645
daily-small40left	standin	15	119	8.2	506 / 8797 / 8797	536 / 8846 / 8846
overload	standin	145	317	29.4	357 / 11690 / 14390	408 / 11713 / 14397
halved-static	standin	29	256	7.4	632 / 3819 / 11566	652 / 3826 / 11585
burst-nocache	standin	62	249	16.0	1001 / 9991 / 15049	1016 / 10027 / 15060
overload-naive	standin	171	315	34.6	0 / 10298 / 12937	0 / 10311 / 12957
halved	standin	29	256	7.4	688 / 7580 / 10045	709 / 7600 / 10058
daily	standin	15	119	8.2	622 / 8837 / 8837	644 / 8887 / 8887
halved-static-naive	standin	19	215	5.9	697 / 20181 / 20181	704 / 20219 / 20219

Table 2b: Outcomes, same rows

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers
steady-real	100.0	0.0	0.0	0.36	1	2424	$0.43
steady	96.3	3.7	0.0	0.15	1	1553	$0.28
burst	59.1	34.8	6.1	0.03	3	933	$0.16
daily-small40left	93.3	6.7	0.0	0.13	1	1035	$0.12
overload	79.3	13.1	7.6	0.01	3	788	$0.11
halved-static	69.0	0.0	31.0	0.34	2	1147	$0.17
burst-nocache	38.7	50.0	11.3	0.06	3	1664	$0.26
overload-naive	71.3	12.3	16.4	0.02	3	677	$0.10
halved	79.3	13.8	6.9	0.14	2	1201	$0.17
daily	46.7	53.3	0.0	0.13	1	969	$0.13
halved-static-naive	73.7	5.3	21.1	0.53	1	1553	$0.24

Table 3: Retrieval settings (real, 25 questions each)

Setting (piece size, pieces fetched, re-rank)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
quality-small-k8-rr-sentence-20b (real)	25	52.0	4.0	20.0	2107	1150 / 1912	1150 / 2144
quality-mid-k4-rr-sentence-20b (real)	25	60.0	0.0	20.0	2198	1342 / 3174	1342 / 3174
quality-mid-k4-norr-sentence-20b (real)	25	64.0	4.0	20.0	2041	1467 / 3343	1487 / 3343
quality-mid-k4-rr-sentence-120b (real)	25	40.0	0.0	20.0	2668	1651 / 12862	1707 / 12862
quality-large-k2-norr-sentence-20b (real)	25	60.0	4.0	20.0	1854	1181 / 7156	1183 / 7156
quality-mid-k4-rr-group-120b (real)	25	44.0	0.0	20.0	2355	1545 / 12786	1616 / 12786
quality-mid-k4-rr-group-120b_citefix (real)	25	60.0	0.0	20.0	1997	1335 / 3082	1335 / 3158

Refused wrongly % per setting:

- quality-large-k2-norr-sentence-20b: 16.0% (n=25), errors/non-ok: 0
- quality-mid-k4-norr-sentence-20b: 12.0% (n=25), errors/non-ok: 0
- quality-mid-k4-rr-group-120b: 36.0% (n=25), errors/non-ok: 0
- quality-mid-k4-rr-group-120b_citefix: 20.0% (n=25), errors/non-ok: 0
- quality-mid-k4-rr-sentence-120b: 40.0% (n=25), errors/non-ok: 0
- quality-mid-k4-rr-sentence-20b: 20.0% (n=25), errors/non-ok: 0
- quality-small-k8-rr-sentence-20b: 24.0% (n=25), errors/non-ok: 0

Excluded from tables (laptop lost network; HTTP status 0, request never reached Groq): quality-large-k2-norr-20b-netfail (25 rows: {‘error’: 20, ‘ok’: 4, ‘rate_limited’: 1}), quality-mid-k4-rr-120b-netfail (8 rows: {‘error’: 8})
Cost uses assumed Groq paid prices of $0.15/$0.60 (120b) and $0.075/$0.30 (20b) per million input/output tokens. I have not checked these; on the

free tier the actual cost is $0. Peak queue length comes from the governor's own counter (`results/peaks.jsonl`), because the CSV cannot reconstruct it. The excluded rows are listed under the tables and stay in `results.csv`.